Your Sovereign Digital Space — Setting Up PeaceOS on Digital Ocean

A Peace Engineers Client Guide: Cloud-First Onboarding for Individuals, Families & Organizations

---

Introduction

Most people believe that reclaiming their digital life means buying new hardware, wiping a laptop, and hoping nothing breaks. It doesn't have to start that way. Before you touch a single piece of hardware, you can experience full digital sovereignty from any browser, on any device you already own.

This guide walks you through standing up a complete **PeaceOS** environment in the cloud — a personal, private computer running **NixOS** with the full **Peace Protocols** intelligence layer, reachable from your Windows PC, your Mac, your Linux box, your iPhone, or your Android phone. No local installation. No commitment to a new operating system on day one. Just a web address, a password, and your own sovereign space waiting for you.

We call this the **cloud-first path**, and it exists because the hardest part of any migration is the fear of the unknown. When you can *log in and use* your sovereign system for a few weeks first — living in it, trusting it, watching your own numbers come alive — the eventual move to physical hardware stops being a leap of faith and becomes a simple, confident next step.

Think of it as two phases:

- **Phase 1 — Cloud Virtual PeaceOS (this guide).** A Digital Ocean droplet becomes your always-on sovereign computer, accessible from anywhere.
- **Phase 2 — Hardware Migration (optional, covered separately).** When you're ready, you copy your configuration onto a physical NixOS/PeaceOS machine and carry your entire environment with you — unchanged.

This guide is written for clients of **Peace Engineers**, and for anyone following the open-source peace-protocols repository. You don't need to be a system administrator. You need only the willingness to follow along — we'll explain not just *what* to do, but *why* each step matters.

Part 1: Understanding the Architecture

Before we build anything, let's make sure you understand what you're building. Sovereignty you don't understand isn't sovereignty — it's just a different landlord. So here's the whole picture, in plain language.

What is a Digital Ocean Droplet?

A **droplet** is simply a computer that lives in a data center instead of on your desk. Digital Ocean rents you a slice of a powerful server — some processing power, some memory, some disk — and gives you complete control over it, as if it were a machine sitting in your own home. You reach it over the internet. It runs 24 hours a day. And unlike a shared account on someone else's platform, *you* decide what runs on it, what it stores, and who can reach it.

The analogy that helps most people: a droplet is **your personal cloud computer**. Not a folder on someone else's computer — an actual machine that answers to you alone.

What is NixOS / PeaceOS?

NixOS is a Linux operating system with one remarkable property: the entire system is described in a single text file. Everything installed, every service running, every setting — all of it is declared in plain language in one place. Change the file, run one command, and the machine becomes exactly what the file says. Nothing hidden, nothing accumulating in the dark.

This gives you three things no ordinary system can:

- **Reproducibility** — the same file produces the same machine, every time, on any hardware.
- **Atomic rollbacks** — every change is a new "generation" you can instantly return to if something goes wrong.
- **No hidden state** — nothing installs itself behind your back; if it's running, you can point to the line that put it there.

PeaceOS is NixOS with a purpose — our opinionated configuration that layers the Peace Protocols intelligence system on top of that rock-solid base.

What is Peace Protocols?

Peace Protocols is the intelligent layer that makes your sovereign machine genuinely useful. It's a constellation of **20 AI agents**, coordinated by a master orchestrator named **Raven**, each responsible for one dimension of a sovereign life — your energy, water, food, health, shelter, knowledge, finances, community, and more. Beneath the agents runs a **Unified MCP Bus** that wires together **23 integrated services** (inference, voice, intelligence, commerce, and more), with local AI inference as the primary backend so your thinking never has to leave your own machine.

The Three Layers

Your sovereign environment is built in three clean layers, each answering to the one above it:

```
        [ Your Browser / Any Device ]
        Windows · Mac · Linux · iPhone · Android
```

```
                    |
                    |  HTTPS (encrypted)
                    v
  [ Digital Ocean Droplet — NixOS / PeaceOS ]
    |
    +-- Peace Protocols Dashboard (web UI)
    +-- MCP Bus (23 integrated services)
    +-- Agent Constellation (20 agents, led by Raven)
    +-- Your Sovereign Data (Spaces bucket)
```

Infrastructure (Digital Ocean) hosts the machine. **The operating system** (NixOS/PeaceOS) makes it reproducible and secure. **The intelligence** (Peace Protocols) makes it serve you. That's the whole stack.

Why Digital Ocean Specifically?

We recommend Digital Ocean for the cloud-first path for a few honest reasons. It's **simple** — the interface is clean and doesn't require a cloud-engineering degree. It's **affordable** — a capable environment starts around $18/month, and a testing setup as low as $8.80/month. Its **Spaces object storage** is excellent and inexpensive for holding your data. There's **no lock-in** — everything we set up is portable, and you can leave whenever you like. And it has a clean, open **API** that lets Peace Engineers automate provisioning for clients who'd rather we handle the whole setup. You are never trapped, and that is precisely the point.

Part 2: Provisioning Your Droplet

Let's build your machine. If you'd prefer we do this for you, Peace Engineers offers full white-glove provisioning — but everything here is designed so you can do it yourself and understand each choice.

Step 1: Create a Digital Ocean Account

Go to digitalocean.com, click **Sign up**, and verify your email. You'll add a payment method — Digital Ocean bills only for what you use, prorated by the hour. New accounts frequently qualify for promotional credit (often **$200 free for 60 days**), which is more than enough to run your entire cloud-first phase at no cost while you decide whether this life is for you.

Step 2: Create Your Project

In the left sidebar, click **New Project**. Give it a meaningful name — something like `peace-sovereign`, or your family or organization name. Projects are organizational buckets that group your resources together. For Peace Engineers managing multiple clients, each client gets their own project; for a single household, one project holds everything.

You'll land on a project page much like the default `first-project` that new accounts start with — a clean dashboard listing your droplets and storage buckets.

Step 3: Provision a Droplet

Click the green **Create** button at the top and choose **Droplets**. Now you'll make a handful of choices:

- **Region.** Pick the data center closest to where you physically are, for the snappiest experience. `SFO3` (San Francisco) is ideal for the US West Coast; `NYC3` for the East Coast; `LON1`, `FRA1`, `SGP1`, and others serve Europe and Asia. Latency to your own machine matters, so choose thoughtfully.
- **OS Image.** Choose **Ubuntu 24.04 (LTS) x64**. This may seem surprising — we're going to run NixOS, after all — but Digital Ocean doesn't offer NixOS as a native image, so we start from Ubuntu and convert it in Part 3. This is the standard, well-trodden path.
- **Droplet Size.** This is the most important choice, so here's honest guidance:
  - **Testing only:** 1 vCPU / 1 GB RAM / 25 GB SSD — **$8.80/month**. This is the size of the reference droplet many clients start with. It runs, but Peace Protocols will feel cramped.
  - **Recommended for a single person:** 1 vCPU / 2 GB RAM / 50 GB SSD — **~$18/month**. This is the comfortable floor for a real, daily-use single-user environment.
  - **For organizations:** 2 vCPU / 4 GB RAM / 80 GB SSD — **~$36/month** and up. More on sizing in Part 7.
- **Authentication.** Choose **SSH Key** if you can — it's far more secure than a password. Digital Ocean shows you exactly how to paste in your public key. (On Mac or Linux, run `ssh-keygen` in a terminal, then `cat ~/.ssh/id_ed25519.pub` to see the key to paste. On Windows, use PuTTYgen or the built-in OpenSSH.) If you're not comfortable with keys yet, a strong password works for now and we'll harden it later.
- **Backups.** Enable **Automated Daily Backups**. On the base plan this adds roughly **$1.60/month**, and it is worth every cent — it's a nightly safety net for your entire sovereign environment. Many clients' droplets show "Last backup was 16 hours ago" right on the overview page, and that quiet reassurance is exactly what you want.
- **Hostname.** Name it meaningfully: `peaceos-yourname` or `peaceos-orgname`.

Click **Create Droplet**. In about sixty seconds your machine is alive, and its dashboard shows an **Active** status, a **Public IPv4** address (for example, `164.90.146.122`), a private IP, and your monthly cost. Copy that public IP — you'll need it constantly.

Step 4: Create a Spaces Bucket (Your Sovereign Data Store)

Your droplet has disk space, but for documents, agent memory, backups, and workflow outputs, we use **Spaces** — Digital Ocean's S3-compatible object storage. It's cheap, durable, and cleanly separated from the machine itself, so your *data* survives even if you rebuild the *computer*.

Click **Create → Spaces Object Storage**. Choose the **same region** as your droplet, and give the bucket a meaningful name — for example, `yourname-peace-data`. (You may already have one; many clients run a bucket like `imaginefreedom` holding their early files.)

Then generate access keys: in the API section of the Digital Ocean console, create a **Spaces access key** and **secret**. **Save both immediately and privately** — the secret is shown only once. You'll paste these into your Peace Protocols configuration so your agents can read and write your sovereign data store.

Part 3: Converting Ubuntu to NixOS (nixos-infect)

Here's the pivotal step: turning that stock Ubuntu droplet into a NixOS machine. We use a well-established community tool called **nixos-infect**, which converts a running Ubuntu or Debian system into NixOS *in place*. It's widely used, well-tested, and the standard way to run NixOS on providers that don't offer it natively.

A gentle warning before we begin: this process **replaces the operating system**. That's the point — but it means we take a safety net first.

Step 1: Connect to Your Droplet via SSH

From your computer's terminal (Terminal on Mac/Linux, PowerShell or Windows Terminal on Windows):

```
ssh root@YOUR_DROPLET_IP
# for example:
ssh root@164.90.146.122
```

If you set up an SSH key, you'll connect straight in. If you chose a password, you'll be prompted for it. The first time, you'll be asked to confirm the server's fingerprint — type `yes`.

Step 2: Take a Snapshot First

This is your undo button. In the Digital Ocean dashboard, open your droplet, go to **Backups & Snapshots → Take Snapshot**, and name it `pre-nixos-infect`. If anything goes sideways during conversion, you can restore this snapshot and be back to a clean Ubuntu in minutes. Never skip this.

Step 3: Run nixos-infect

Back in your SSH session, first update the system and install the couple of tools the script needs:

```
apt-get update && apt-get install -y curl git
```

Now run nixos-infect. We pin it to the current stable NixOS channel and tell it to reboot into NixOS when finished:

```
        curl https://raw.githubusercontent.com/elitak/nixos-infect/master/nixos-
infect | \
          NIX_CHANNEL=nixos-24.11 \
          NIXOS_IMPORT=./host.nix \
          bash 2>&1 | tee /tmp/nixos-infect.log
```

What's happening: the script installs the Nix package manager, builds a complete NixOS system alongside the running Ubuntu, rewrites the bootloader to point at NixOS, and reboots. It takes roughly **5 to 15 minutes**. Your SSH session will freeze and then disconnect when the machine reboots — that's expected, not a failure. The `tee` command saves a full log to `/tmp/nixos-infect.log` in case you ever need to review it.

Step 4: Reconnect After Reboot

Wait two or three minutes for the reboot to complete, then reconnect:

```
        ssh root@YOUR_DROPLET_IP
```

You may get a warning that the host key changed — that's correct, because the operating system genuinely changed. Remove the old key with `ssh-keygen -R YOUR_DROPLET_IP` and reconnect. Once you're in, confirm your success:

```
        nixos-version
        # should print something like: 24.11.xxxxxxx (Vicuña)
```

If you see a NixOS version, congratulations — your cloud machine is now running NixOS.

Step 5: A Clean Base Configuration

nixos-infect generates a working configuration, but let's replace it with a clean, purposeful one. Open the system's single source of truth:

```
        nano /etc/nixos/configuration.nix
```

Here's a solid starting configuration. It opens the ports Peace Protocols needs, enables SSH, installs the essentials, and creates a normal user account:

```
        { config, pkgs, ... }:
        {
```

```
  imports = [ ./hardware-configuration.nix ];

  networking.hostName = "peaceos";
  networking.firewall.allowedTCPPorts = [ 22 80 443 8000 8001 ];

  services.openssh.enable = true;
  services.openssh.settings.PermitRootLogin = "yes";  # tighten after setup

  environment.systemPackages = with pkgs; [
    git curl wget
    python312 python312Packages.pip python312Packages.virtualenv
    htop vim tmux
  ];

  users.users.peace = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];       # allows sudo
    initialPassword = "changeme";    # change this immediately after first
login
  };

  system.stateVersion = "24.11";
}
```

Save the file (in nano: `Ctrl+O`, `Enter`, then `Ctrl+X`) and apply it:

```
nixos-rebuild switch
```

That single command reads your file and *becomes* the machine it describes. From now on, this file **is** your computer. Back it up, version-control it, and you can recreate this exact environment anywhere, forever.

Part 4: Installing Peace Protocols

Now we bring the intelligence layer online. There are two paths: a straightforward manual install to get you running immediately, and the elegant NixOS-native install in Part 5 that makes everything permanent and reproducible. We'll start manual so you can see each piece working.

Step 1: Clone the Repository

```
git clone https://github.com/peaceengineer0001/peace-protocols.git
cd peace-protocols
```

Step 2: Install the Python Dependencies

Because this is NixOS, we enter a shell that provides Python and its tooling, then create an isolated virtual environment:

```
nix-shell -p python312 python312Packages.pip python312Packages.virtualenv

python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
# install the project itself so its packages are importable everywhere:
pip install -e .
```

Step 3: Configure Your Environment

Peace Protocols never ships live secrets. Instead, it provides `.example.toml` templates you copy and fill in with your own values:

```
cp config/peace-protocols.example.toml config/peace-protocols.toml
cp config/llm-providers.example.toml   config/llm-providers.toml

nano config/peace-protocols.toml
```

In these files you'll set your droplet's IP or domain, and — importantly — your **Spaces bucket name and the access keys** you saved in Part 2, so your agents can use your sovereign data store. These live-config files are deliberately excluded from git (via `.gitignore`), so your secrets never leave your machine.

Step 4: Validate the MCP Registry

Before starting anything, confirm the 23-server registry parses cleanly:

```
python3 scripts/validate_mcp_registry.py
# Expected:
# Loaded 23 servers (6 native, 17 adapters)
# 0 error(s), 0 warning(s)
```

Zero errors means every service is correctly wired in the registry.

Step 5: Start the MCP Bus

The MCP Bus is the nervous system that connects every agent to every tool:

```
python3 -m mcp_bus.serve
```

You'll watch the connection pool bring servers up concurrently, health-checking each one. Services whose upstream daemon isn't provisioned yet (a GPU inference backend, an external API key) will report offline — and that's expected on a fresh droplet. The bus **isolates faults**, so one missing service never takes down the rest.

Step 6: Start the Peace Protocols Dashboard

The dashboard is your window into the whole system — a polished web UI showing system health, the agent constellation, live workflows, and interactive calculators for the Peace and Community Vitality indexes. Install its dependencies and launch it bound to all interfaces so you can reach it from your browser:

```
pip install fastapi uvicorn pyyaml httpx
python3 -m uvicorn app:app --host 0.0.0.0 --port 8000
```

Now open a browser on *any* device and visit `http://YOUR_DROPLET_IP:8000`. There it is — your sovereign dashboard, live.

You can preview exactly what your dashboard will look like right now, before you build anything, at the Peace Engineers reference deployment: **peace-navy.abacusai.cloud**. Your own dashboard will be identical, but running on *your* machine, serving *your* data.

Part 5: Making It Permanent (systemd + the NixOS Flake)

Running commands by hand is fine for a first look, but a sovereign system should start itself on boot and survive reboots without you babysitting it. NixOS makes this beautiful.

The systemd approach

The repository already includes systemd service definitions in `nixos/modules/`. The MCP Bus module (`nixos/modules/mcp-bus.nix`) declares a proper service:

```
systemd.services.peace-mcp-bus = {
  wantedBy = [ "multi-user.target" ];    # start on boot
  serviceConfig = {
    ExecStart = "${pkgs.python313}/bin/python3 -m mcp_bus.serve";
    Restart   = "on-failure";
  };
};
```

Because it's declared this way, the bus starts automatically at boot, restarts if it ever crashes, and is captured in your configuration forever.

The NixOS flake approach (recommended)

The elegant path is to let NixOS manage the *entire* Peace Protocols stack declaratively. The repo's `nixos/` directory contains a `flake.nix` and six modules — `peace-protocols`, `mcp-bus`, `inference`, `voice`, `intel`, and `commerce`. From the repository directory on your droplet:

```
        cd nixos
        nix flake show                          # inspect what the flake offers
        sudo nixos-rebuild switch --flake .#peaceos # build & switch to the full
PeaceOS system
```

That one command wires the MCP Bus, the agents, and every service you've enabled into your system as first-class, auto-starting components. To turn capabilities on or off, edit the module options — for example `services.peaceProtocols.mcpBus.enable = true;` or the inference and voice toggles — and rebuild. Every capability is **opt-in**, and your whole environment is now described in code you own.

Part 6: Web Browser Access — The Key to Hardware-Free Onboarding

This is the heart of the cloud-first promise: **you reach your entire sovereign system from a web browser, on any device, with nothing installed locally.**

Right now your dashboard answers at `http://YOUR_DROPLET_IP:8000`. That already works from your laptop, your tablet, and your phone. But let's make it polished, secure, and memorable.

Point a domain at your droplet (recommended)

A raw IP address is forgettable and can't be secured with a friendly certificate. If you own a domain (or buy one for a few dollars a year), create an **A record** pointing, say, `peace.yourdomain.com` at your droplet's public IP. Digital Ocean can even manage your DNS for you under **Networking → Domains**.

Put nginx in front with HTTPS

We serve the dashboard through **nginx** as a reverse proxy — the same pattern Peace Engineers uses for the reference deployment. On NixOS, this is a few declarative lines plus free, auto-renewing certificates from Let's Encrypt:

```
services.nginx = {
  enable = true;
  virtualHosts."peace.yourdomain.com" = {
    enableACME = true;     # automatic Let's Encrypt certificate
    forceSSL   = true;     # redirect all traffic to HTTPS
    locations."/".proxyPass = "http://127.0.0.1:8000";
  };
};
security.acme = {
  acceptTerms = true;
  defaults.email = "you@yourdomain.com";
};
```

Run `sudo nixos-rebuild switch`, and within moments your dashboard is live at `https://peace.yourdomain.com` — encrypted, trusted by every browser, and reachable worldwide.

Create your "PeaceOS Portal"

Here's a small ritual that makes sovereignty feel real day to day: on each device you use, create a **dedicated browser profile** named "PeaceOS" (Chrome, Firefox, and Edge all support named profiles), and bookmark your dashboard as its home page. That profile becomes your portal — a clean, distraction-free doorway into your sovereign environment that follows you from your work laptop to your phone to a borrowed machine at a library. Same portal, same agents, same data, wherever you are.

On the horizon

Peace Engineers is developing a **custom browser extension and Progressive Web App (PWA)** that will auto-connect to a client's PeaceOS droplet with a single tap — turning any device into a native-feeling PeaceOS terminal, no setup required. Cloud-first clients will be the first to receive it.

Part 7: Use Case Tiers

Sovereignty looks different for a single person than it does for a community of hundreds. Here are four concrete configurations, each with real Digital Ocean sizing, honest monthly costs, the agents that shine, and a picture of daily life. Use them as starting points; Peace Engineers tunes the exact mix for you.

Tier 1 — The Sovereign Individual

- **Droplet:** 1 vCPU / 2 GB RAM / 50 GB SSD — **~$18/month**
- **Spaces:** 250 GB — **~$5/month**
- **Backups:** enabled — **~$3.60/month**

- **All in:** roughly **$27/month**

What's running. The full 20-agent constellation, with the MCP Bus serving inference, voice, and intelligence services. Your finances are watched by **Ember**, your health sovereignty by **Heal**, and your personal knowledge base by **Lore** and **Sage**.

A typical day. You open your PeaceOS portal over morning coffee and read **Raven's** briefing — a synthesis of everything the council surfaced overnight. You dictate a journal entry that **VoxCPM** transcribes in your own voice profile. Over lunch you glance at the **World Monitor** for situational awareness of the things you actually care about, filtered from the noise. In the evening you use **Shopstr** for a peer-to-peer purchase that never touches a corporate marketplace. Every one of these runs on *your* machine, computing *your* numbers.

Access. Dashboard from laptop, tablet, and phone — the same portal everywhere.

Tier 2 — The Sovereign Family

- **Droplet:** 2 vCPU / 4 GB RAM / 80 GB SSD — **~$36/month**
- **Spaces:** 500 GB — **~$10/month**
- **Backups:** enabled — **~$7.20/month**
- **All in:** roughly **$53/month** for the whole household

What's running. A shared agent constellation with per-person personalization — each family member gets their own agent profile and privacy boundary. **Haven** integrates with Home Assistant to give the household a sovereign view of shelter, energy, and devices. Shared calendars and workflows keep everyone in sync, while **Lore** supports the children's learning sovereignty with a family knowledge base that isn't mining your kids for ad revenue.

A typical day. A weekly **family council**, facilitated by the **Council** agent, reviews the household's shared resources — **Root** (food), **Sol** (energy), and **Tide** (water) — and surfaces where the family is growing more self-reliant. The kids use a curated, private learning space; the parents track the household's real resilience instead of a bank's version of it.

Tier 3 — The Sovereign Organization (5–25 people)

- **Droplet:** 4 vCPU / 8 GB RAM / 160 GB SSD — **~$72/month** (or a small Kubernetes cluster as you grow)
- **Spaces:** 1 TB+ — **~$21/month**
- **Backups & extras:** budget **~$15/month**
- **All in:** roughly **$110/month** for the organization

What's running. The full 23-server MCP Bus, put to work. **TryComp** provides sovereign CRM; the **Marketing Agent** runs the Fletcher Method for outreach; **HiveTalk SFU** hosts your video conferencing

on infrastructure you control; **OpenCodeReview** supports your technical team. **Raven** acts as Chief of Staff, orchestrating all 20 domain agents, with **Council** governing decisions, **Forge** overseeing production and manufacturing, and **Thrive** tracking the organization's real economics.

A typical day. Every employee reaches the org's web portal from any device — no per-seat SaaS subscriptions bleeding you monthly. Standups, customer relationships, marketing, and internal knowledge all run through agents that answer to the organization rather than to a vendor. Power users who fall in love with the workflow can later migrate to physical PeaceOS hardware while keeping the identical environment.

Tier 4 — The Sovereign Community Node

- **Droplet:** 8 vCPU / 16 GB RAM — **~$144/month**, plus a dedicated **GPU Droplet** for local inference (billed hourly, spin up as needed)
- **Storage & bandwidth:** budget generously for public traffic and media
- **All in:** varies with GPU usage; a serious node runs a few hundred dollars a month — shared across an entire community

What's running. The complete NixOS flake with all six modules enabled. A **Nostr relay** carries the community's censorship-resistant communications. A **Lightning node (LND)** manages a shared community treasury. **HiveTalk SFU** hosts community-wide video gatherings. A public-facing website welcomes newcomers while a private member dashboard serves those inside. This is sovereignty at the scale of a village — infrastructure a community owns together, coordinated by the same agent constellation that serves a single person, just larger.

A typical day. Members communicate over the community's own relay, transact through the community's own Lightning treasury, meet over the community's own video platform, and make decisions through **Council** — none of it dependent on a corporation that could deplatform them tomorrow.

Part 8: Security & Sovereignty Hardening

A sovereign machine must also be a *defended* one. Once your environment is running, spend twenty minutes on these essentials.

- **SSH keys only.** Once your key works, disable password login entirely. In `configuration.nix`: set `services.openssh.settings.PasswordAuthentication = false;` and tighten `PermitRootLogin` to `"prohibit-password"` (or better, log in as your `peace` user and use `sudo`). Rebuild.

- **A tight firewall.** Open only what you use. For a dashboard-only setup that's ports **22** (SSH), **80** and **443** (web). Close **8000** and **8001** to the public once nginx is fronting the dashboard, since traffic then arrives over 443.
- **HTTPS everywhere.** The Let's Encrypt setup from Part 6 ensures every byte between your browser and your droplet is encrypted and renews automatically.
- **Stay current.** Update your system deliberately with `sudo nixos-rebuild switch --upgrade`. Because NixOS upgrades are atomic and reversible, updating is safe — if anything misbehaves, you roll back from the boot menu.
- **Encrypt and guard your data store.** Enable encryption on your Spaces bucket, and treat your Spaces secret key like a house key — it opens your sovereign data.
- **Back up your Nostr keys.** Your Nostr private key *is* your identity across the federation. Export it, store it offline in at least two safe places, and never paste it into anything you don't control. Lose it and you lose your identity; leak it and someone can impersonate you.
- **Never commit secrets.** Keep every live value in the `config/*.toml` files that `.gitignore` already excludes. The repository ships only `.example.toml` templates — follow that pattern religiously and your secrets stay yours.

Part 9: Migrating to Physical Hardware (When Ready)

The cloud-first phase isn't a detour — it's a dress rehearsal for the real thing, and NixOS makes the transition almost magical.

When you're ready to run PeaceOS on a physical machine — a laptop reclaimed from Windows, a mini PC in your home, a server in your community space — you don't rebuild anything. Your entire droplet is described by two files: `configuration.nix` and the PeaceOS `flake.nix`. You copy those files to your new machine, run `sudo nixos-rebuild switch --flake .#peaceos`, and your physical computer becomes the *same* environment you've been living in — the same agents, the same MCP Bus, the same workflows, the same data (synced from your Spaces bucket) — only now it's local, faster, and entirely in your hands.

There is **zero reconfiguration**. That's the whole promise of declarative infrastructure: your machine is a document, and documents travel.

For the full hardware journey — backing up, building install media, the BIOS steps, and standing PeaceOS up on a physical laptop — see the companion guide, **nixos-asus-zenbook-install.md**. When you make the move, you can keep your cloud droplet running as a backup and remote-access node, or simply cancel it. Either way, your sovereignty comes with you.

Part 10: Troubleshooting Common Snags

Even the smoothest onboarding hits an occasional bump. Here are the questions we field most often, with fixes you can apply in minutes.

"I ran `nixos-infect` and lost my SSH connection." This is expected — the script reboots the droplet at the end. Wait two or three minutes, then reconnect. If your terminal complains about a changed host key (a warning about "REMOTE HOST IDENTIFICATION HAS CHANGED"), that's also normal: the machine genuinely changed operating systems. Remove the old key with `ssh-keygen -R YOUR_DROPLET_IP` and reconnect. If you still can't reach the droplet after five minutes, use the **Recovery Console** from your Digital Ocean dashboard to log in directly and check `journalctl -xb` for boot errors.

"`nixos-rebuild` fails with an error I don't understand." NixOS is refreshingly honest about failures — nothing changes until the build fully succeeds, so a failed rebuild leaves your running system untouched. Read the last few lines of the output; they usually name the exact option or package at fault. Fix the line in `configuration.nix`, save, and rebuild again. Because every prior configuration is preserved, you are never stranded.

"The MCP registry reports errors or warnings." Run `python3 -m mcp_bus.validate` and read the per-server output. The most common cause is a missing value in your `config/*.toml` files — an unset API key or an adapter pointed at a service that isn't running yet. Adapters you don't need can be disabled in `config/scope-config.toml`; the registry only validates what you've switched on.

"The dashboard won't load in my browser." Work outward from the droplet. First confirm the service is alive with `systemctl status peace-dashboard`. Then confirm nginx is running and its certificate issued with `systemctl status nginx`. Finally, confirm your domain's DNS **A record** actually points at your droplet's IP — DNS changes can take up to an hour to propagate. Ninety percent of "it won't load" reports resolve at one of these three checkpoints.

"How much will this really cost me?" Your only unavoidable charge is the droplet itself — from roughly **6–18 per month** depending on size — plus a few cents for Spaces storage and optional backups. There are no per-agent fees, no license costs, and no usage metering inside PeaceOS. You pay Digital Ocean for the computer; everything running on it is yours.

"Can I resize later if I outgrow my droplet?" Yes. Digital Ocean lets you power down, resize to a larger plan, and power back on in minutes — and because your entire system is declared in `configuration.nix`, nothing needs reinstalling. Start small with confidence; you can always grow.

Conclusion

You began this guide on a device you already own, perhaps wondering whether digital sovereignty was something reserved for engineers and enthusiasts. By the end of it, you have — or know exactly how to build — a complete, private, intelligent computer in the cloud, reachable from anywhere, answering to no one but you.

That's the quiet power of the cloud-first path. No leap of faith, no wiped laptop on day one, no fear of the unknown. Just a browser tab, a login, and a sovereign space that grows with you — from a single person finding their footing, to a family sharing resilience, to an organization owning its infrastructure, to a community holding its own treasury and its own voice.

When you're ready to bring it all home to physical hardware, your configuration comes with you, unchanged. And if you'd like a steady hand at any point along the way, that's exactly what we're here for.

Peace Engineers Email: **raven@peaceengineers.org** Code: **github.com/peaceengineer0001/peace-protocols** Live demo: **peace-navy.abacusai.cloud**

Your sovereignty doesn't require permission, just intention — and an $18/month droplet to start.

— Raven Rolland Gregg, Peace Engineers